University for the Creative Arts

BERLIN SCHOOL OF BUSINESS & INNOVATION

Essay / Assignment Title: **Advanced Robotics, Automation and Computer Vision**

Programme title: **MSc in Artificial Intelligence**

Name: **Sanket Sadashiv Gadge**

Year: **2025**

CONTENTS

CONTENTS..... 2
TASK ONE: AUTONOMOUS DRONE-BASED VISION SYSTEM FOR CROP HEALTH MONITORING AND DISEASE CLASSIFICATION USING DEEP LEARNING..... 4
ABSTRACT..... 4
INTRODUCTION..... 5
CHAPTER ONE: LITERATURE REVIEW..... 6
CHAPTER TWO: PROBLEM FORMULATION..... 6
CHAPTER THREE: DATASET PREPARATION..... 7
CHAPTER FOUR: MODEL ARCHITECTURE..... 8
Supervised Classification Experiments..... 8
Contrastive Learning with a Siamese Network..... 9
CHAPTER FIVE: CODE IMPLEMENTATION..... 11
Data loading/preprocessing..... 11
Model Definition..... 13
Training Script:..... 14
CHAPTER SIX: MODEL EVALUATION..... 16
CONCLUDING REMARKS..... 19
TASK TWO: VISION-GUIDED AUTONOMOUS NAVIGATION IN ROS..... 20
INTRODUCTION..... 20
CHAPTER ONE: PROJECT DEFINITION..... 21
CHAPTER TWO: DESIGN AND IMPLEMENTATION..... 22
Tools and Libraries Used..... 22
CHAPTER THREE: EXECUTION AND TESTING..... 24
CHAPTER FIVE: EVALUATION AND REFLECTION..... 27
CONCLUDING REMARKS..... 28
BIBLIOGRAPHY..... 29
APPENDIX (if necessary)..... 31

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the program).

Name and Surname (Capital letters):

SANKET GADGE

.....

Date: 01/10/2025

TASK ONE: AUTONOMOUS DRONE-BASED VISION SYSTEM FOR CROP HEALTH MONITORING AND DISEASE CLASSIFICATION USING DEEP LEARNING

ABSTRACT

Agriculture is very important for any country's economy. Crop diseases are a big problem for farmers which can reduce the food supply. To help with this, we need fast and accurate ways to find these diseases early. In this paper, we tried various deep learning models to find 17 diseases in five crops: corn, potato, rice, sugarcane, and wheat. This study is about making and comparing scalable computer vision models for finding five types of crop diseases using a public dataset from Kaggle. Our main purpose was to compare different CNN models. We used three famous models—ResNet18, MobileNetV2, and EfficientNetB2—and tried different ways to train them, like fine-tuning, freezing layers, and training from zero. We also tried a different method called deep metric learning. For this, we used a Siamese network with a ResNet18 backbone. This model doesn't just classify, it learns to see how similar a healthy leaf is to a diseased one. After training and testing all the models, we found that the ResNet18 finetuned model gave the best result, achieving a top-1 accuracy of 92.32%. For comparison, the other models performed differently; Siamese classifier with Resnet18 backbone gave 75.78% accuracy , Finetuned MobileNetV2 got 87.63% , and Finetuned EfficientNetB2 achieved 83.53% accuracy. This comparison clearly shows that the ResNet18 model is the superior choice for accurately identifying diseases in this dataset. We also found that all models had trouble with some diseases. This is because the dataset has some classes with very few images and some diseases look very similar to each other. Our results show that when you pick a model, you have to look at accuracy, but also at model size and speed.

Keywords: **Convolutional Neural Networks (CNNs)**; **Image Classification**; **Transfer Learning**; **Finetuning, ResNet, MobileNet, EfficientNet, Siamese Networks.**

INTRODUCTION

Plant diseases cause big losses for farms all over the world. Every year, we lose 10% to 40% of major crops, which is billions of dollars. (Mamun et al., 2024) Finding these diseases early is very important to manage crops well. It helps to use the right pesticides only where needed and saves the food we grow. Right now, farmers or experts have to check plants by hand. New technology in farming, called precision agriculture, needs automatic tools that can find diseases using data (Kumar and Rao, 2024). We can use computers to process images and find plant diseases using Computer Vision. It's called Fine-Grained Visual Classification (FGVC), which means telling apart things that look very similar. This is hard for two reasons. One being that disease can look different depending on how old it is or the weather (intra-class variance). And another being that different diseases can look the same, with similar spots or colors (inter-class similarity). Also another big problem is the data we have. In the real world, there are many photos of common diseases but very few photos of rare ones. This is called class imbalance (Albattah and Khan, 2025)

Nowadays, deep learning, especially Convolutional Neural Networks (CNNs), has become the SOTA way to classify images, also including plant diseases. In transfer learning you take a model that was already trained on a huge dataset like ImageNet (Deng et al., 2009) and then you train it a little more on your specific problem. This works really well even if you don't have a huge dataset.

CHAPTER ONE: LITERATURE REVIEW

Many researchers have worked on using AI for finding plant diseases. Most studies use deep learning models like CNNs. For example, some papers show that models like ResNet and VGG can get very high accuracy, sometimes over 98%, on datasets like PlantVillage (Li, Zhang and Wang, 2021). These studies prove that deep learning is very powerful for this task. Other research focuses on using AI with drones for farming. Drones with cameras can cover large fields much faster than a person can. The AI models on these drones can analyze the images in real-time to find problems (Parker et al., 2025). This helps in what is called "precision agriculture," where farmers can give water or pesticides only to the specific plants that need it. (Padhiary et al., 2024) This saves money and is better for the environment.

The gap I found is that most papers focus on only one thing. They either try to get the highest accuracy with big models, or they focus only on making a lightweight model. Not many papers compare all these different types of models (high accuracy, lightweight, and metric learning) on the same dataset.

CHAPTER TWO: PROBLEM FORMULATION

The main goal of our project is to build a computer vision system that can be put on a drone to find crop diseases from the sky. To do this, we have to solve a few problems.

First, the system must be very accurate. If it makes mistakes, farmers might use the wrong treatment or miss a disease, which could ruin the crops. Meaning we need to reduce the number of False Positives and Negatives.

Second, the system must be fast and lightweight. A drone has a small computer and runs on a battery. A big, slow AI model will not work. It will drain the battery too fast and won't be able to check the images in real-time as the drone flies. So, we need a model that is small in size (MB) and has a faster inference speed (ms).

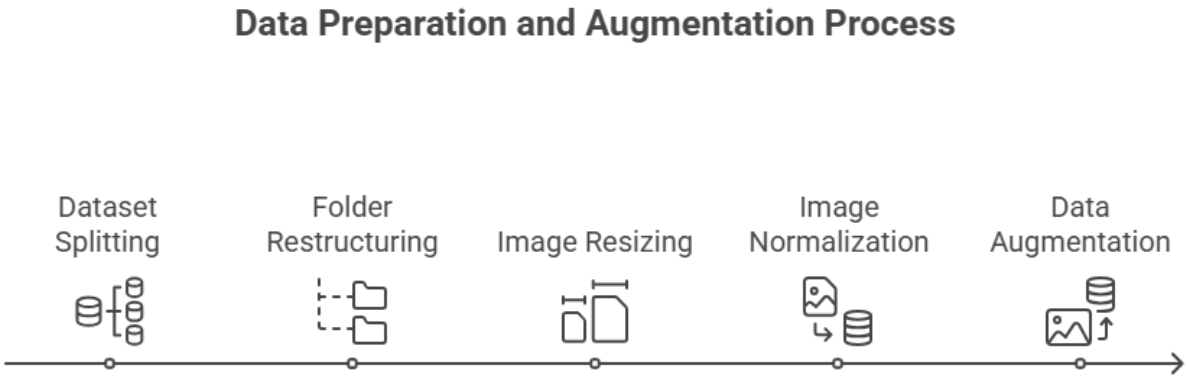
Third, the system must handle the challenges of real-world data. Like Imbalance, The system needs to work well even for rare diseases. It also has to deal with images that might be low quality / blurry images.

CHAPTER THREE: DATASET PREPARATION

We used the "Five Crop Diseases Dataset" from Kaggle (Sharma, 2022). This dataset is huge and also has a high usability score on Kaggle, because it has 13,324 photos of 17 different classes, for five important crops: corn, potato, rice, sugarcane, and wheat. It has photos of healthy leaves and leaves with diseases like rust, blight, and spots. A big thing about this dataset is that the classes are not balanced. Some diseases have almost 1,500 photos, but others have only 100. You can see the number of images for each class in Table 1 below.

Table 1: Dataset Class Distribution

Class Name 1	Num Images	Class Name 2	Num Images
Corn___Common_Rust	1192	Corn___Gray_Leaf_Spot	513
Corn___Healthy	1162	Corn___Northern_Leaf_Blight	985
Potato___Early_Blight	1000	Potato___Healthy	152
Potato___Late_Blight	1000	Rice___Brown_Spot	613
Rice___Healthy	1488	Rice___Leaf_Blast	977
Rice___Neck_Blast	1000	Sugarcane___Bacterial_Blight	100
Sugarcane___Healthy	100	Sugarcane___Red_Rot	100
Wheat___Brown_Rust	902	Wheat___Healthy	1116
Wheat___Yellow_Rust	924		



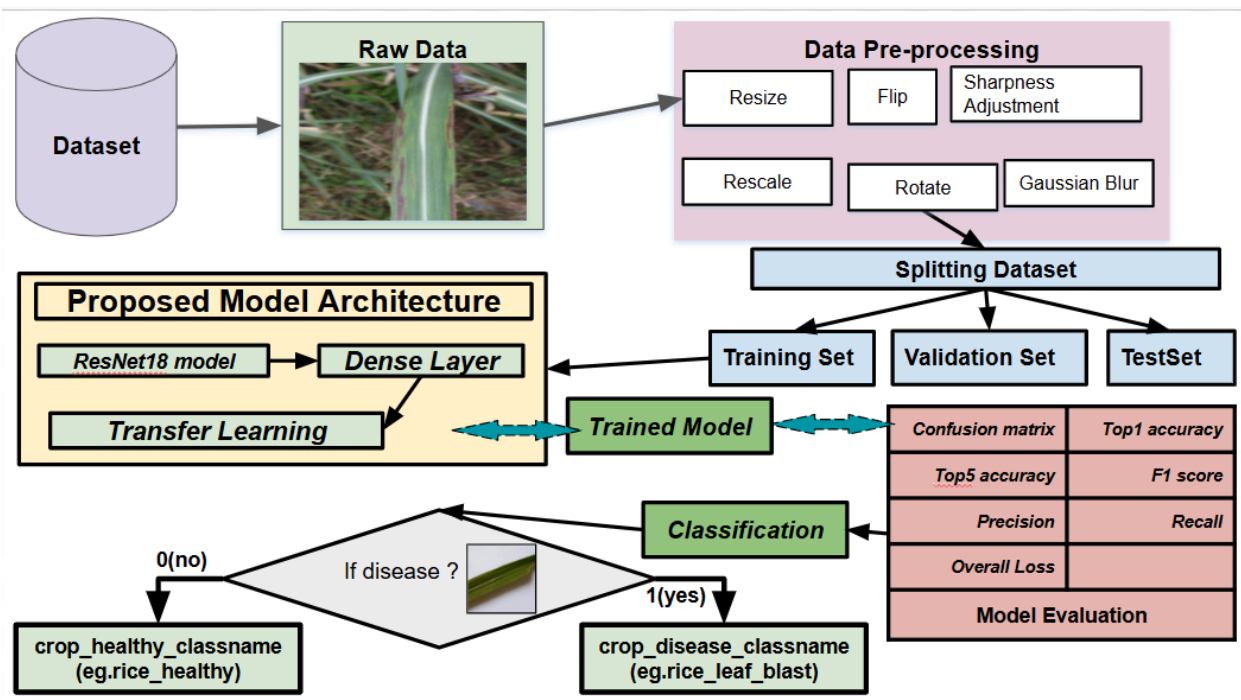
CHAPTER FOUR: MODEL ARCHITECTURE

Supervised Classification Experiments

We picked three different types of CNNs to get a good idea of what works best:

- **ResNet18**: A classic model that is known to be very accurate. It uses special connections called "residual connections" that help it learn better (He et al., 2016).
- **MobileNetV2**: A small and fast model made for phones and other small devices. It's designed to be efficient (Sandler et al., 2018).
- **EfficientNetB2**: A modern model that is supposed to be very good at balancing accuracy and speed (Tan and Le, 2019).

Fig 1: Proposed Crop Disease classification Framework



Here’s our model architecture of the fine-tuned ResNet18 model which achieved the highest accuracy.

Stage	Resolution	Channels	Operator	No. of Layers
1	-	17	Linear	1
2	1 × 1	512	AdaptiveAvgPool2d	1
3	112 × 112	64	BatchNorm2d	1
4	112 × 112	64	Conv2d	1
5	112 × 112	64	ReLU	1
6	14 × 14	256	BatchNorm2d	5
7	14 × 14	256	Conv2d	5
8	14 × 14	256	ReLU	4
9	28 × 28	128	BatchNorm2d	5
10	28 × 28	128	Conv2d	5
11	28 × 28	128	ReLU	4
12	56 × 56	64	BatchNorm2d	4
13	56 × 56	64	Conv2d	4
14	56 × 56	64	MaxPool2d	1
15	56 × 56	64	ReLU	4
16	7 × 7	512	BatchNorm2d	5
17	7 × 7	512	Conv2d	5
18	7 × 7	512	ReLU	4

Table 2. ResNet18_FineTuned Layer Architecture

Contrastive Learning with a Siamese Network

We also tried a different approach called deep metric learning. We used a Siamese network. This is basically two identical CNNs that share the same weights. This network doesn't give a class name. Instead, it takes a photo and turns it into a vector (a list of numbers) in an "embedding space." The goal is to make the vectors for similar photos very close to each other, and vectors for different photos very far from each other (Schroff, Kalenichenko and Philbin, 2015).

Triplet Margin Loss

We trained this network using something called Triplet Margin Loss (Schroff, Kalenichenko and Philbin, 2015). In each step, we give the network three photos: an "anchor" (A), a "positive" (P) which is the same class as the anchor, and a "negative" (N) which is a different class. The loss function tries to make

the distance between A and P small, and the distance between A and N big. The formula for the loss (L) is:

$$L(A,\ P,\ N)\ =\ max(\ ||f(A)\ -\ f(P)||_2\text{^}2\ -\ ||f(A)\ -\ f(N)||_2\text{^}2\ +\ \alpha\ ,\ 0\ )$$

Here, **f(·)** is our **Siamese network**, and **α** is a "**margin**," which is a number that says how far apart the pairs should be. We used a margin of **1.0.**

CHAPTER FIVE: CODE IMPLEMENTATION

Here are some important parts of the Python code I wrote in PyTorch.

Data loading/preprocessing

```
data_dir = "dataset"

train_transforms = transforms.Compose([

    transforms.Resize((img_size, img_size)),

    transforms.RandomAdjustSharpness(sharpness_factor=2),

    transforms.RandomHorizontalFlip(),

    transforms.RandomRotation(degrees=(0, 180)),

    transforms.GaussianBlur(kernel_size=(3, 5), sigma=(0.1, 5.)),

    transforms.RandomPerspective(distortion_scale=0.2, p=1.0),

    transforms.ToTensor(),

    transforms.Normalize([0.485, 0.456, 0.406],

                         [0.229, 0.224, 0.225])

])

val_test_transforms = transforms.Compose([

    transforms.Resize((img_size, img_size)),

    transforms.ToTensor(),

    transforms.Normalize([0.485, 0.456, 0.406],

                         [0.229, 0.224, 0.225])

])
```

```
train_dataset = datasets.ImageFolder(root=f"{data_dir}/train",
transform=train_transforms)

val_dataset   = datasets.ImageFolder(root=f"{data_dir}/val",
transform=val_test_transforms)

test_dataset  = datasets.ImageFolder(root=f"{data_dir}/test",
transform=val_test_transforms)

train_loader = DataLoader(train_dataset, batch_size=batch_size,
shuffle=True, num_workers=num_workers, pin_memory=True,
persistent_workers=True, prefetch_factor=4)

val_loader   = DataLoader(val_dataset, batch_size=batch_size,
shuffle=False, num_workers=num_workers, pin_memory=True,
persistent_workers=True, prefetch_factor=4)

test_loader  = DataLoader(test_dataset, batch_size=batch_size,
shuffle=False, num_workers=num_workers, pin_memory=True,
persistent_workers=True, prefetch_factor=4)

print("Classes:", train_dataset.classes)

print("Train size:", len(train_dataset))

print("Val size:", len(val_dataset))

print("Test size:", len(test_dataset))
```

Model Definition

```python
import torch
model = torch.hub.load('pytorch/vision:v0.10.0', 'resnet18', pretrained=True)
_ = model.eval()
# count the number  of trainable parameters
num_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Number of trainable parameters: {num_params}")
```

```
Downloading: "https://github.com/pytorch/vision/zipball/v0.10.0" to /root/.cache/t
/usr/local/lib/python3.11/dist-packages/torchvision/models/_utils.py:208: UserWarn
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/torchvision/models/_utils.py:223: UserWarn
  warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /root/
100%|██████████| 44.7M/44.7M [00:00<00:00, 162MB/s]
Number of trainable parameters: 11689512
```

```python
for param in model.parameters():
    param.requires_grad = False

# Unfreeze last convolutional block
for param in model.layer4.parameters():
    param.requires_grad = True
num_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Number of trainable parameters: {num_params}")
```

```
Number of trainable parameters: 8393728
```

Here, I have initialized the resnet18 model with pretrained=True which means I am getting the model along with the ImageNet weights. Further I have frozen the pretrained model except for the last few layers. This is done because we want to use the knowledge that the model has already learnt and train on top of that, which is called finetuning.

Training Script:

```python
def train(self, epochs):
    for epoch in range(epochs):
        # Training loop
        self.model.train()
        train_loss, correct, total = 0, 0, 0

        loop = tqdm(self.train_loader, desc=f"Epoch [{epoch+1}/{epochs}]")
        for images, labels in loop:
            images, labels = images.to(self.device), labels.to(self.device)

            outputs = self.model(images)
            loss = self.criterion(outputs, labels)

            self.optimizer.zero_grad()
            loss.backward()
            self.optimizer.step()

            train_loss += loss.item() * images.size(0)
            _, preds = outputs.max(1)
            correct += preds.eq(labels).sum().item()
            total += labels.size(0)

            loop.set_postfix(loss=loss.item(), acc=100. * correct / total)

        epoch_loss = train_loss / total
        epoch_acc = 100. * correct / total
        self.writer.add_scalar("Loss/train", epoch_loss, epoch)
        self.writer.add_scalar("Accuracy/train", epoch_acc, epoch)

        # Validation loop
        val_loss = self.validate(epoch)

        if val_loss == -1:
            break
        if self.scheduler:
            if isinstance(self.scheduler, lr_scheduler.ReduceLROnPlateau):
                self.scheduler.step(val_loss)  # pass monitored metric
            else:
                self.scheduler.step()
```

In training, we loop over the batches of images and labels, we do a forward pass through our neural network, then getting the loss for that batch, we take an optimizer step and eventually do a backward pass. We accumulate the loss over all the batches as well as the count for the correct and incorrect prediction done in the training step. Finally we log the scalers like train loss and accuracy to tensorboard and go for validation step.

Validation script:

```python
def validate(self, epoch):
    self.model.eval()
    val_loss, correct, total = 0, 0, 0

    with torch.no_grad():
        for images, labels in self.val_loader:
            images, labels = images.to(self.device), labels.to(self.device)
            outputs = self.model(images)
            loss = self.criterion(outputs, labels)

            val_loss += loss.item() * images.size(0)
            _, preds = outputs.max(1)
            correct += preds.eq(labels).sum().item()
            total += labels.size(0)

    if val_loss < self.best_loss:
        self.best_loss = val_loss
        if torch.cuda.device_count() > 1:
            torch.save(self.model.module, self.best_path)
        else:
            torch.save(model, self.best_path)
        print(f"Model Saved at {self.best_path}")
        self.no_improvement = 1 # reset the patience
    else:
        self.no_improvement += 1
        if self.no_improvement >= self.patience:
            print("Validation Loss did not improve for last 5 epochs!")
            print("Early stopping triggered!!!")
            return -1
```

In validation step, we do the same thing as of training but on a different set of images, which were not the part of training.

CHAPTER SIX: MODEL EVALUATION

The results from our eight experiments showed big differences in performance. You can see the Top-1 and Top-5 accuracy for all models in Image 6 and Table 3.

The fine-tuned models were much, much better than the others. **ResNet18_Finetuned** was the best, with **92.32%** Top-1 accuracy and **99.93%** Top-5 accuracy. **MobileNetV2_Finetuned** was next with **87.63%** Top-1, and then **EfficientNetB2_Finetuned** with **83.53%** Top-1. The models where we froze the main part were very bad. Their accuracies were only around **18-22%**. This means the features learned from ImageNet are not good enough for plant diseases without more training. The **ResNet18_Scratch** model, trained from zero, was the worst with only **3.80%** accuracy. This really shows that transfer learning is super important for this kind of dataset.

Model	F1_Score	Precision	Recall	Top1_accuracy	Top5_accuracy	Inference_Time_Ms
ResNet18_Finetuned	**0.901**	**0.911**	**0.901**	**0.923**	**0.999**	9.598
MobileNet-V2_Finetuned	0.845	0.857	0.848	0.876	0.999	5.900
EfficientNetB2_FineTuned	0.758	0.814	0.775	0.835	0.996	76.964
ResNet18_Siamese_Classifier	0.506	0.508	0.548	0.758	0.961	10.761
MobileNetV2_Frozen	0.146	0.126	0.177	0.221	0.376	26.278
EfficientNetB2_Frozen	0.160	0.157	0.173	0.218	0.478	29.211
ResNet18_Frozen	0.142	0.154	0.145	0.181	0.510	**3.526**
ResNet18_Scratch	0.006	0.003	0.058	0.038	0.241	9.870

Table 3. Comparative analysis of ResNet18, EfficientNetB2, MobilenetV2, Siamese classifier (Triple loss)

Per Class Accuracies for our best performer ResNet18_FineTuned Model:

class_idx	class_name	precision	recall	f1	support
0	Corn___Common_Rust	0.992	0.983	0.987	120
1	Corn___Gray_Leaf_Spot	0.940	0.904	0.922	52
2	Corn___Healthy	0.983	1.000	0.992	117
3	Corn___Northern_Leaf_Blight	0.958	0.929	0.944	99
4	Potato___Early_Blight	1.000	0.990	0.995	100
5	Potato___Healthy	0.842	1.000	0.914	16
6	Potato___Late_Blight	0.990	0.960	0.975	100
7	Rice___Brown_Spot	0.765	0.419	0.542	62
8	Rice___Healthy	0.836	0.887	0.861	150
9	Rice___Leaf_Blast	0.627	0.747	0.682	99
10	Rice___Neck_Blast	1.000	0.990	0.995	100
11	Sugarcane__Bacterial Blight	0.875	0.700	0.778	10
12	Sugarcane__Healthy	1.000	0.900	0.947	10
13	Sugarcane__Red Rot	0.750	0.900	0.818	10
14	Wheat___Brown_Rust	0.948	1.000	0.973	91
15	Wheat___Healthy	0.983	1.000	0.991	113
16	Wheat___Yellow_Rust	1.000	1.000	1.000	93

Table 4. ResNet18_FineTuned classification report (test)

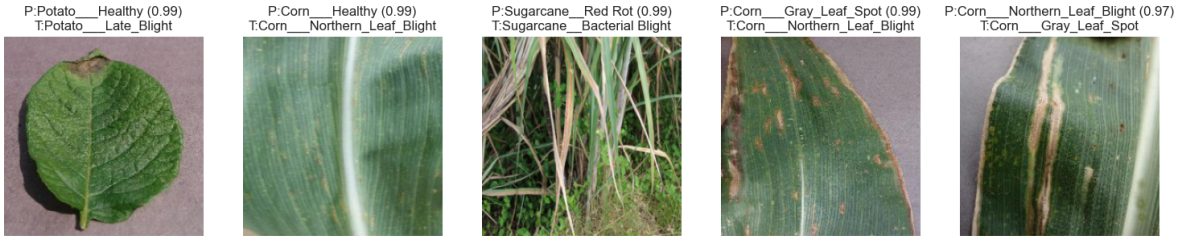


Fig 2: Incorrect prediction by Resnet18_Finetuned

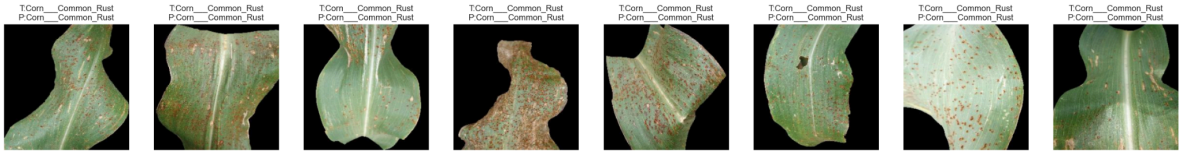


Fig 3: correct prediction by Resnet18_Finetuned

The big challenge is probably not the models, but the dataset itself. There are two main reasons. One being the classes with bad scores are the ones with less data. You can see in Table 1 that the three Sugarcane classes only has 100 images each.

Fig 4: Heat map of per class F1-score for each model

How Did the Siamese Network Do?

The ResNet18_Siamese_Classifier was our different approach. It got 75.78% accuracy. This is not as good as the fine-tuned models, but it's way better than the normal ResNet18_Frozen model, which only got 18.11%. This is a very important result because both models used the exact same frozen ResNet18. The only difference was how the features were learned. This shows our Siamese network, with the special Healthy vs. Diseased triplet training, was very good at learning useful features for this problem.

CONCLUDING REMARKS

In this project, we did a deep check of different deep learning models for finding crop diseases from a drone. By comparing eight models, we found a clear picture of what works best. The results show that a fine-tuned ResNet18 gives the highest accuracy, so it is good for powerful, server-based systems. But for use in the field on a real drone, a fine-tuned MobileNetV2 is a better choice because it balances accuracy and efficiency. We also showed that deep metric learning with a Siamese network is a good idea for learning special features from the data. The biggest challenge we found is that all models have trouble with the same few diseases, which is because of problems in the dataset. So, for the future, we need to focus on both making better models and using better data strategies.

TASK TWO: VISION-GUIDED AUTONOMOUS NAVIGATION IN ROS

INTRODUCTION

For task two, we are applying computer vision principles to achieve Vision-Guided Autonomous Navigation inside a controlled simulation environment. The main objective of Task 2 is to develop autonomous mobile robot simulation, using the Robot Operating System (ROS) platform within “TheConstruct.ai”. This task addresses the essential problem of how mobile robots (like simulating a TurtleBot3) can perceive their environment, which requires building a functional vision module. We must use a robot's RGB camera sensor for doing object detection on a defined target, like some colored cube or marker. Crucially, this vision data is necessary for calculating the object's relative position and then applying movement logic, commanding the robot to either move forward or rotate to center the target.

CHAPTER ONE: PROJECT DEFINITION

The main problem in this task was to use the camera sensor and robot in ROS2 for detecting color and moving according to detection.

Steps I did:

- First I set up robot simulation in ROS2 using a cloud system.
- Then I used a camera for getting live video.
- After that, I wrote a program to detect one specific color from the camera feed.
- Finally, I moved the robot according to the position of that color. If color comes on the left side, the robot will turn left. If color comes on the right side, the robot will turn right. If color is in the center, the robot will go forward. If no color is detected, then the robot will stop.

We did this project on TheConstruct.ai cloud platform. This platform is helpful because it already has an ROS environment ready. So we don't need to install ROS in our own laptop.

- **Simulation World**: We used Gazebo simulator. The world we opened is turtlebot3_house.launch.py. This is a small house with rooms and furniture. In this world, we kept one green box to be our target object.
- **Robot**: The robot we used is TurtleBot3 Waffle Pi. It is a small two-wheel robot, very common for ROS learning. It has a front camera and LIDAR sensor. But in this project, we only used the RGB camera.
- **Software**: We used the ROS2 Humble framework. All code was written in Python 3, using the **rclpy** library.

The name of our ROSject on TheConstruct.ai is **S5VNP4YsDT**.

CHAPTER TWO: DESIGN AND IMPLEMENTATION

Architecture Overview

Our system has a very simple architecture. It is all inside one ROS2 node called color_detector. This node does everything: it gets the images, processes them, and sends commands to the robot's wheels.

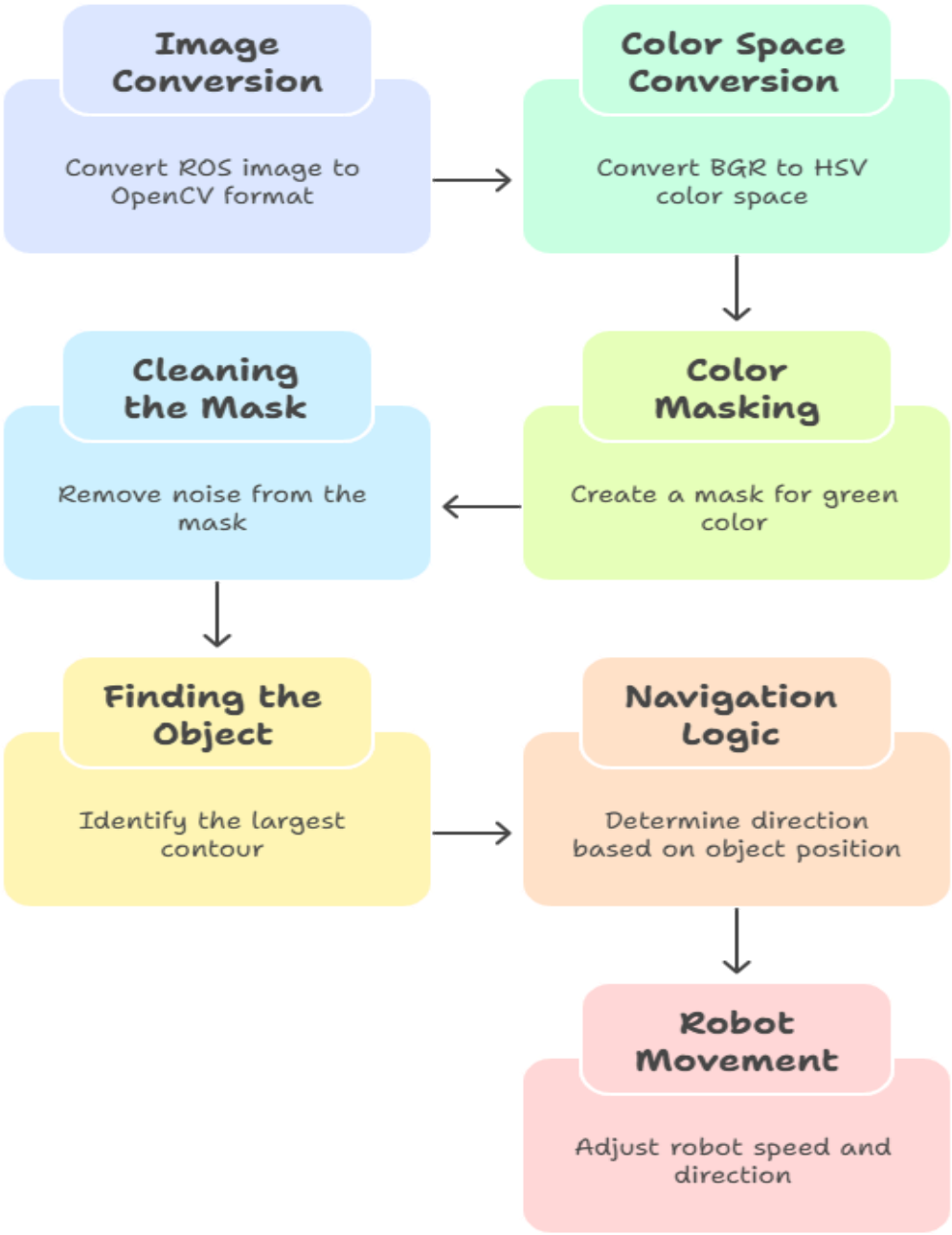
Here is the flow:

- The Camera on the TurtleBot3 publishes images to the /camera/image_raw topic.
- Our color_detector node subscribes to this topic to get the images.
- For each image, the node uses OpenCV to check for the color green.
- Based on where the green object is in the image, the node decides which way to move (left, right, forward, or stop).
- The node then publishes a Twist message to the /cmd_vel topic. This topic controls the linear (forward/backward) and angular (turning) speed of the robot's wheels.

Tools and Libraries Used

- **ROS2 Humble:** The main framework for our robot software.
- **rclpy:** for creating ROS2 nodes, publishers, and subscribers.
- **Gazebo:** The simulator for the robot and the world.
- **sensor_msgs:** This package provides the Image message type for camera data.
- **geometry_msgs:** provides the Twist message type for sending velocity commands.
- **Python 3:** main program
- **OpenCV (cv2):** for computer vision and image processing.
- **NumPy:** Used for handling the image data as arrays.

Color Detection and Navigation Process



CHAPTER THREE: EXECUTION AND TESTING

Test Cases and Outcomes

To run the project, we first built our ROS2 workspace using

```
>> colcon build.
```

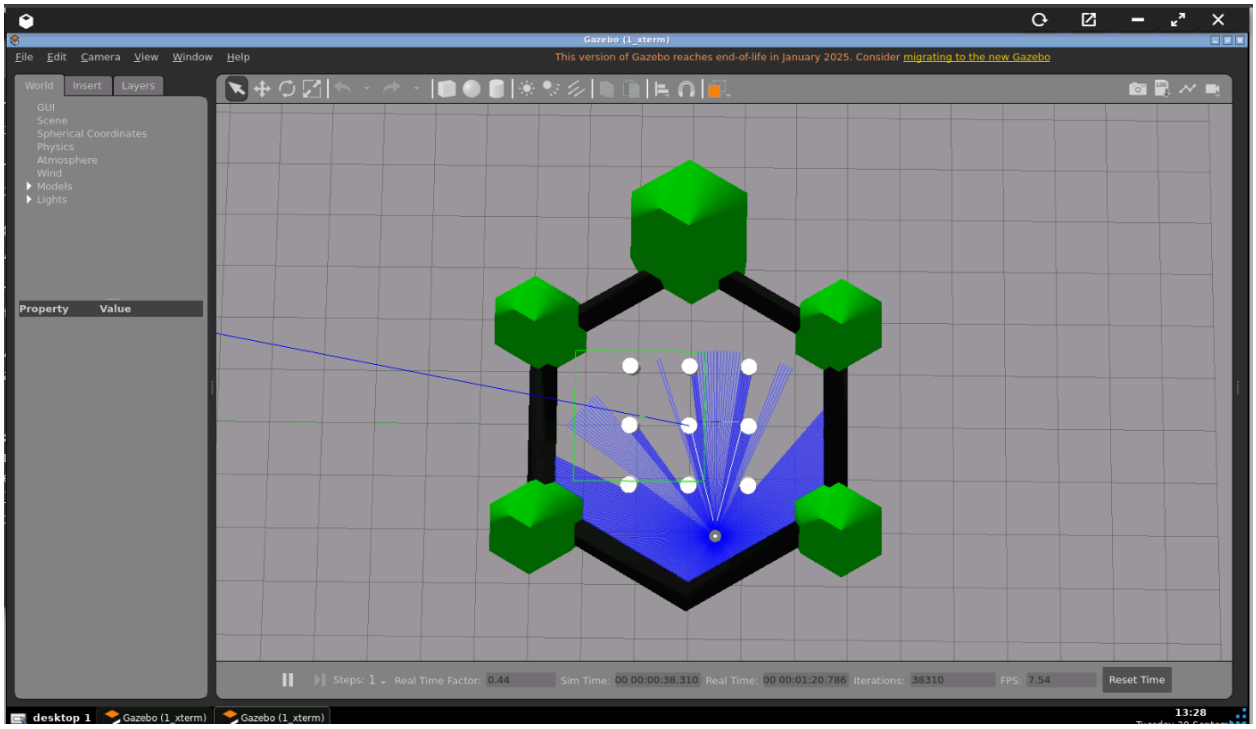
Then, we launched our simulation world and the robot using our custom launch file color_detection_bot.launch.py. Then, in a new terminal, we ran our Python node with

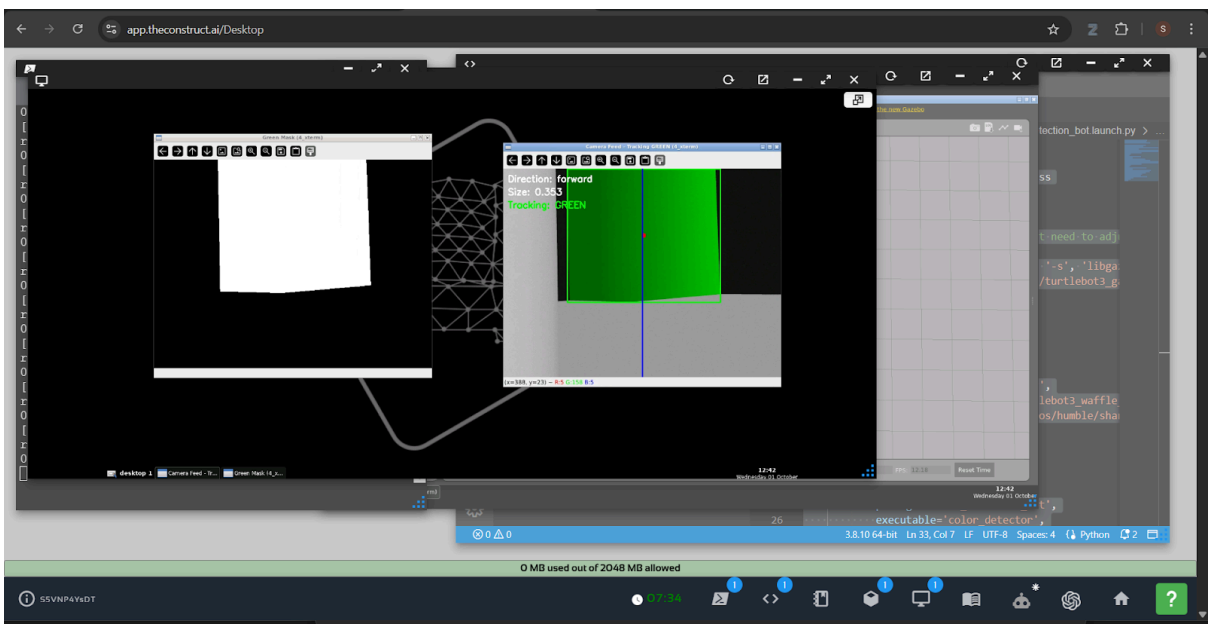
```
>> ros2 run color_detection_bot color_detector
```

We tested the robot with a few different situations:

- **Test Case 1: Object in the center.** We placed the green box directly in front of the robot. The robot correctly identified it, and the camera feed window showed "Direction: forward". The robot started moving straight towards the box.
- **Test Case 2: Object to the left.** We placed the box to the robot's left. The robot saw it on the left side of its camera view. The direction changed to 'left', and the robot started turning left until the box was in the center. Then it started moving forward.
- **Test Case 3: Object to the right.** This was the same as the left test, but the robot turned right.
- **Test Case 4: No object.** When we moved the box out of the robot's view, the robot stopped as expected. The direction showed 'stop'.

The screenshots below show the system working. The first image is the Gazebo world with the TurtleBot3 and the green box. The second image is the camera feed window from our Python script. You can see the green box has a green rectangle drawn around it, and the text on top shows the direction is 'forward' and the robot is tracking 'GREEN'.





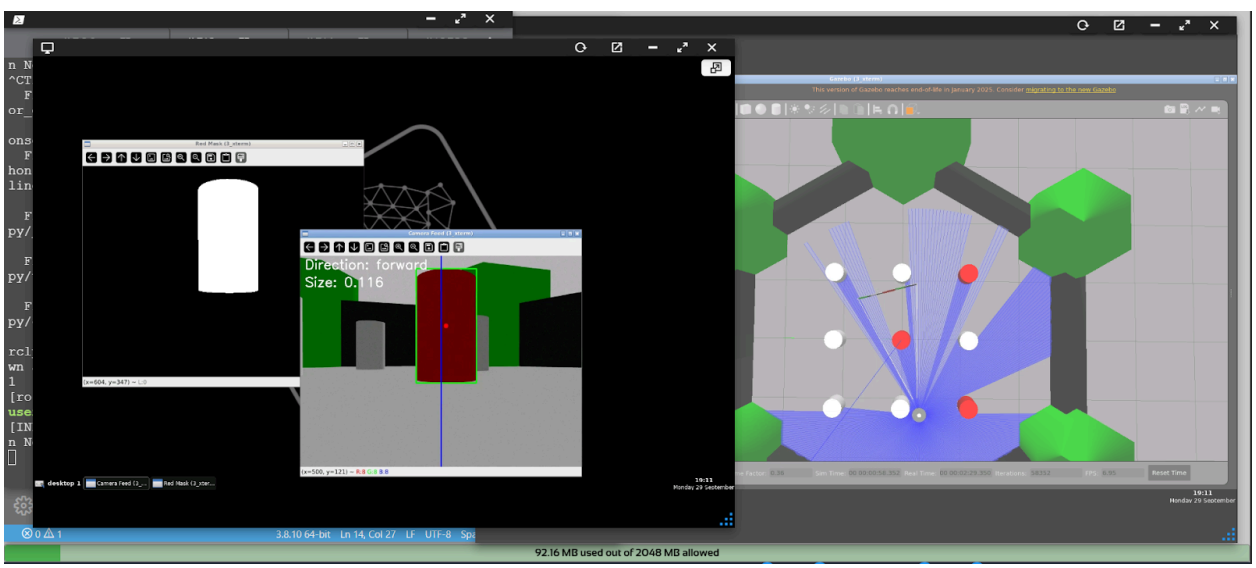
The tests were successful. The robot behaved exactly as we programmed it to.

CHAPTER FIVE: EVALUATION AND REFLECTION

Challenges Faced and Solutions

We faced a few small challenges during this project:

- **Finding the Camera Topic**: In ROS, you need to know the exact name of the topic to get data from it. We were not sure what the TurtleBot3's camera topic was. We tried a few common names and finally found that /camera/image_raw worked. Our code now prints a log message to show which topic it is looking for.
- **Tuning HSV Values**: Finding the perfect HSV range for the color green in Gazebo was a bit of trial and error.
- **Persistent environment:** As we were using cloud based RosJect projects in [theconstruct.ai](), every time i used to close the project and open it, i used to loose my custom configuration made to the turtlebot world. Below you can see, i tried to used red color for detection and also made changes to color of the pillars to make them red. But it lost the changes next time i opened the project after closing.



Potential Improvements:

- **Obstacle Avoidance:** Right now, the robot will crash into the green box or any other obstacle in its way. The TurtleBot3 has a LIDAR sensor. We could add logic to use the LIDAR data to stop the robot if it gets too close to something.
- **CNN BasedDetection:** Simple color detection is not very reliable. If the lighting changes a lot, or if there are other green things in the room, the robot can get confused. We could use a more advanced detection method, like training a simple CNN model (like we did in Task 1) to recognize the specific shape of the box, not just its color.
- **More colors:** The color 'green' is hardcoded in our program. A good improvement would be to make the target color a ROS parameter. This way, we could tell the robot to follow a blue or red object just by changing a parameter when we launch the node, without changing the code.

Reflection on Real-World Robotics

This project was a great introduction to understand how robots could see the world. This could be useful in scenes, For example, a warehouse robot could follow a person wearing a radium vest. In agriculture, a robot could follow a row of green plants to spray water or check for diseases. This project taught us how to connect perception to action using ROS2.

CONCLUDING REMARKS

In conclusion, this task was a success. We built a robot in ROS2 that uses its camera and OpenCV to find and follow a green object. This project showed us the basic, but important, steps of how a real robot can see the world and move on its own. It's a good foundation for making more complex robots in the future.

BIBLIOGRAPHY

1. Barman, S., Farid, F.A., Jaohar Raihan, Khan, N.A., Ferdous, M., Bhattacharya, A., Mahmud, Z., Sadia Afrin Ridita, Sarker, M.T., Karim, H.A. and Mansor, S. (2024). Optimized Crop Disease Identification in Bangladesh: A Deep Learning and SVM Hybrid Model for Rice, Potato, and Corn. *Journal of Imaging*, [online] 10(8), pp.183–183. doi:https://doi.org/10.3390/jimaging10080183.
2. He, K., Zhang, X., Ren, S. and Sun, J. (2015). *Deep Residual Learning for Image Recognition*. [online] Available at: https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf.
3. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. [online] arXiv.org. Available at: https://arxiv.org/abs/1801.04381.
4. Tan, M. and Le, Q.V. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. [online] arXiv.org. Available at: https://arxiv.org/abs/1905.11946.
5. Jafar, A., Bibi, N., Rizwan Ali Naqvi, Abolghasem Sadeghi-Niaraki and Jeong, D. (2024). Revolutionizing agriculture with artificial intelligence: plant disease detection methods, applications, and their limitations. Frontiers in Plant Science, 15. doi:https://doi.org/10.3389/fpls.2024.1356260.
6. Mamun, A.A., Ahmedt-Aristizabal, D., Zhang, M., Md Ismail Hossen, Zeeshan Hayder and Awrangjeb, M. (2024). Plant Disease Detection Using Self-supervised Learning: A Systematic Review. IEEE Access, [online] 12, pp.171926–171943. doi:https://doi.org/10.1109/access.2024.3475819.
7. Kumar, S.P. and Rao, Y.R. (2024). A Survey on Deep Learning Approaches for Crop Disease Analysis in Precision Agriculture. Turkish Journal of Computer and Mathematics

Education (TURCOMAT), [online] 15(1), pp.242–253. doi:https://doi.org/10.61841/turcomat.v15i1.14699.

8. Albattah, W. and Khan, R.U. (2025). Impact of Imbalanced Features on Large Datasets. Frontiers in Big Data, 8. doi:https://doi.org/10.3389/fdata.2025.1455442.
9. Schroff, F., Kalenichenko, D. and Philbin, J. (2015). FaceNet: A Unified Embedding for Face Recognition and Clustering. In: *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [online] Boston, MA, USA: IEEE, pp.815–823. Available at: https://arxiv.org/abs/1503.03832.
10. Sharma, S. (2022). *Five Crop Diseases Dataset*. [online] Kaggle. Available at: https://www.kaggle.com/datasets/shubham2703/five-crop-diseases-dataset.
11. Li, L., Zhang, S. and Wang, B. (2021). Plant Disease Detection and Classification by Deep Learning—A Review. *IEEE Access*, 9, pp.56683–56698. doi:https://doi.org/10.1109/access.2021.3069646.
12. Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K. and Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp.248–255. doi:https://doi.org/10.1109/cvpr.2009.5206848.
13. Parker, T.A., Burcu Celebioglu, Watson, M. and Gepts, P. (2025). Drone methods and educational resources for plant science and agriculture. *bioRxiv (Cold Spring Harbor Laboratory)*. doi:https://doi.org/10.1101/2025.05.19.654604.
14. Padhiary, M., Saha, D., Kumar, R., Sethi, L.N. and Kumar, A. (2024). Enhancing precision agriculture: A comprehensive review of machine learning and AI vision applications in all-terrain vehicle for farm automation. Smart Agricultural Technology, [online] 8, p.100483. doi:https://doi.org/10.1016/j.atech.2024.100483.
15. Jean, G. and Ailyn, D. (2025). Lightweight CNN Models for Edge Deployment in Low-Resource Environments. [online] ResearchGate. Available at: https://www.researchgate.net/publication/395767938_Lightweight_CNN_Models_for_Edge_Deployment_in_Low-Resource_Environments.

APPENDIX (if necessary)

1. Link for the dataset used for Task1 - https://www.kaggle.com/datasets/shubham2703/five-crop-diseases-dataset
2. Github Link for the Task 1 https://github.com/Sanket758/AtBSBI/tree/master/Machine%20Learning%20Fundamentals%20-%20Assignment
3. Document with commands for Task2 - https://docs.google.com/document/d/1oArwgh2JgQ8XqppHHYF4MIxRFskdb5T8gjWZ2iV46sk/edit?usp=sharing